# Static Malware Analysis using ELF features for Linux based IoT devices

Akshara Ravi
*Computer Science and Engineering*
*Indian Institute of Technology, Palakkad, India*
*112004002@smail.iitpkd.ac.in*

Vivek Chaturvedi
*Computer Science and Engineering*
*Indian Institute of Technology, Palakkad, India*
*vivek@iitpkd.ac.in*

***Abstract*—With the growing deployment of Internet of Things (IoT) devices in diverse domains, malware authors have started using these devices as attack vectors for distributed attacks targeting critical computing infrastructures. Since IoT devices are highly resource-constrained, traditional malware analysis techniques are usually ineffective to mitigate new and unknown malware threats. In this paper, we propose a novel, fast, and resource-efficient malware detection methodology that makes use of machine learning and focuses on detecting zero-day malware targeting Linux OS. Our approach extracts static features from the Linux Executable and Linkable Format (ELF) executables and applies the chi-square feature selection technique to reduce the number of features, without impacting the overall accuracy. We have evaluated our approach using 7 machine learning models including J48, JRip, PART, Random Forest, Naive Bayes, Logistic, and RIDOR. Compared to other state-of-the-art works, time taken to train these models was very less. The experimental results show that our proposed methodology can achieve an accuracy of more than 99% with less than 0.1% false positive and false negative rate.**

***Index Terms*—ELF, chi-square, feature selection, malware, IoT, Linux, resource-constrained, accuracy, machine-learning.**

## I. Introduction

Malware (malicious software) is defined, generally, as "any code added, changed, or removed from a software system in order to intentionally cause harm or subvert the intended function of the system" [1]. Malware has become a serious threat to the everyday life of people since it impacts all sectors of daily life such as banking, social media, e-commerce, and healthcare. There has been an exponential growth of malware in the recent years, registering over 450,000 new malicious samples per day putting enterprises and businesses under constant threat [2]. Many malware detection techniques were developed in order to safeguard the computer systems and servers from these attacks. The most common technique is deploying commercial antivirus scanners. They detect malware based on signatures available in their database, however fail to detect zero-day malware and polymorphic and metamorphic versions of existing malware [3] [4] [5].

Traditionally, malware authors were interested in developing malware targeted for personal computers and servers. Therefore, most of the malware detection approaches are designed without considering constraints such as memory foot-print, power consumption, and network resources [3] [6] [7]. However, Mirai-based DDoS attack was an eye-opener for malware authors that IoT devices can be used as an effective attack vector targeting critical infrastructure. The IoT malware Mirai caused the biggest DDoS (Distributed Denial-of-Service) attack which targeted the Dyn DNS (Domain Name Service) servers and disabled major internet services such as Amazon, Twitter, and Netflix [8]. As a result, a host of new malware using Mirai's leaked source code and targeting IoT devices have emerged such as Satori, Reaper, Amnesia, and Masuta [9]. These attacks exploit the inherent weaknesses of IoT devices and networks. On several fronts, IoT network is distinct from other distributed systems and therefore have its own unique set of challenges [10]:

- Resource constraints (power, network bandwidth, memory)
- Heterogeneity of processor architecture
- Lack of security design (open ports, default credentials, hard to patch)

From the malware detection perspective of IoT devices, maintaining and updating a huge virus database similar to traditional computer systems is not feasible. Moreover, once the malware gets into a host machine, it can use the files or network resources to spread them. By the time the antivirus scans the whole system and detect the malware, malware would have created havoc using these IoT devices. Hence, most of the recent research work in malware detection is focused on detecting the malware as fast as possible before it can release its malicious payload.

Additionally, a large number of IoT devices run Linux as their OS (approximately 71.8% [11]) due its open-source nature, security, and stability, with distributions such as Raspbian, Ubuntu, and Debian. However, for unknown reasons, there is very limited work available in the literature on protecting Linux based IoT systems from malware [12]. Therefore, securing these resource-constrained devices which are constantly vulnerable to ever-growing malware continues to be a significant challenge [10] [13].

Apart from commercial antivirus solutions, malware detection approaches are primarily categorized into two, namely static and dynamic malware detection [4]. Dynamic detection essentially requires setting up a sandbox environment, executing the malware samples, and monitoring the system calls invoked by the execution. This process is resource-intensive

and time-consuming that makes dynamic analysis not suitable for malware detection in IoT systems [10]. Among the static analysis techniques, converting the malware binaries to gray-scale image, constructing function call graph and opcode graph are complex and resource-intensive tasks [10]. Disassembling binaries and extracting opcodes is time-consuming and opcodes could be easily obfuscated by attacker's simple techniques [10].

Nowadays, malware authors are trying to evade malware detection techniques by adding obfuscation to the executables. Obfuscation changes the structure of the executable significantly, keeping the functionality intact. All the above mentioned approaches are vulnerable to obfuscation. But in the case of ELF header features, adding more obfuscation to the ELF format may destroy its format, or maim the malicious behavior [10]. Hence we have chosen ELF header features since they are easy to extract and amount of obfuscation which can be applied to these features is very limited.

In this paper, we propose a novel, fast, accurate, and resource-efficient static malware detection methodology targeting resource-constrained embedded systems running Linux OS and deployed in IoT networks. The main contributions of our proposed work are enlisted below:

- Proposed a malware detection framework for resource-constrained IoT devices.
- Aims to reduce the number of features that need to be extracted from the executables by applying chi-square feature selection method. Lesser the number of features, lesser the time taken to build the model and predict the class of the executable.
- Reduces the time and memory consumption required to detect whether an executable is malicious or not.
- Conducted extensive experiments using seven different machine learning models by simulating a resource-constrained setup in a virtual machine.
- Achieved an accuracy of around 99% and was able to predict the maliciousness in less than 20 seconds.

The rest of the paper is organized as follows. Section II briefly summarizes the literature review. Section III discusses the proposed methodology. Section IV presents the experiments and results obtained. Section V presents the conclusion.

## II. Literature Review

Due to the increasing count and complexity of the new malware samples which are getting created every day, research work in the area of malware detection has increased by leaps and bounds. Most research works can be categorized based on analysis techniques such as static, dynamic, and hybrid to detect zero-day malware and its obfuscated versions.

Linux OS has gained immense popularity due to its open-source nature, high security, and stability. As a result, malware attacks targeting the Linux environment have grown significantly. Shahzad et al., presented ELF-Miner which do the static malware analysis of Linux executables [6]. However, this work uses more than 100 features from the ELF headers for malware detection and is time and resource-consuming. Therefore, it is not suitable for embedded or IoT devices in which time and space are critical design constraints. Asmitha et al., extract system calls dynamically from the ELF executables using system call tracer *strace* and achieve a classification accuracy of 97% [3]. They also proposed a non-parametric statistical approach using machine learning techniques for identifying previously unknown malicious ELF files and achieved an overall classification accuracy of 97.30% [7]. Both these works log the system calls for just 1 minute and therefore it fails to detect the malware which activate its malicious behavior later. Also, the dataset consists of just 226 Linux malware samples. Damri et al., provided a survey of dynamic analysis techniques for Linux systems [14]. Shahzad et al., proposed a concept of genetic footprint by mining the information in the kernel process control blocks (PCB) of a process in Linux OS [15]. Bai et al., proposed a malware detection approach which mines system calls from the symbol table of Linux executables [16]. This work only dumps the dynamic symbol table (*.dynsym*) section of the ELF format and do feature selection on the system calls present in this section. The *.dynsym* only contains global symbols, not the entire symbol table (*.symtab*).

Since Linux is the popular OS for IoT devices, attacks against Linux-based IoT devices are also on the rise [17]. Due to the heterogeneity in the processor architecture, malware detection in IoT devices is a difficult task. Recent research focuses on security threats of IoT [18] [19] and static and dynamic analysis of malware in IoT devices. Ngo et al., presented a survey on the state-of-the-art static IoT malware detection methods and carried out exactly the methods of existing studies based on the same IoT malware dataset [10]. Raju et al., provided a survey on the cross-architectural IoT malware detection and classification approaches and discussed the feature representations, feature extraction techniques, and machine learning models employed in the surveyed works [13]. Lee et al., proposed that printable strings can be used as an effective cross-platform feature to identify IoT malware on different IoT platforms [20]. They were able to achieve an accuracy of 99% on the same platform and 96% on different platforms with a 120K ELF executable dataset. Since an executable may have a large number of printable strings, extracting those features and converting them to numerical vectors is resource-intensive and not suitable for IoT networks. Wan et al., proposed approaches to detect and classify executable programs using byte sequences [21] [22]. These works use the byte sequences at the entry points, defined in *e_ entry* field of the ELF header to detect whether an ELF file is malware. It is possible that the actual entry point is present at a later section due to obfuscation. Tien et al., proposed a framework that can analyze IoT malware using ELF and opcode features in a wide range of environments [23]. For extracting the opcode features, the binary needs to be disassembled first and would require tools like IDA Pro. Also, obfuscation techniques like instruction substitution replace one opcode with a set of equivalent opcodes [24] and manage to evade the malware detection system.

Recently, Kumar et al., proposed EDIMA, a distributed

modular solution which can be used towards the detection of IoT malware network activity in large-scale networks (e.g. ISP, enterprise networks) during the scanning/infecting phase rather than during an attack [9]. There are works on IoT malware detection that use neural network models. For example, Kumar et al., build a model to tackle the malware detection problem in IoT using the raw byte-sequences which eliminates the need for domain-specific knowledge [25]. The proposed system can classify the IoT binaries as malware or benign with an accuracy of 99.01%. Jeon et al., proposed a dynamic analysis for IoT malware detection (DAIMD) in a nested cloud environment to extract behaviors related to memory, network, virtual file system, process, and system call [26]. By converting the extracted and analyzed behavior data into images, the behavior images of the IoT malware are classified and trained in the Convolutional Neural Network (CNN). Dynamic execution and deep learning models require heavy computational resources. In short, malware detection approaches which use dynamic analysis, large number of static features, and deep learning models for classification are not suitable for resource-constrained IoT devices. Some recent works have also reported fileless attacks which doesn't use any malware files and hence leave no traces on the infected system [27]. Although there are lot of research focused on malware detection, not much work is available in the literature on IoT malware detection. Besides Linux OS, there are several works available on defending Windows OS [5] [24] [28]. In this paper, our focus is on Linux OS as it is the most widely used OS for IoT devices.

## III. Proposed Methodology

Our proposed framework extracts static features from the ELF executables and classify them as benign or malicious. This involves 3 phases: (1) Feature selection, (2) Training and (3) Detection. The block diagram of the framework is shown in Figure 1.

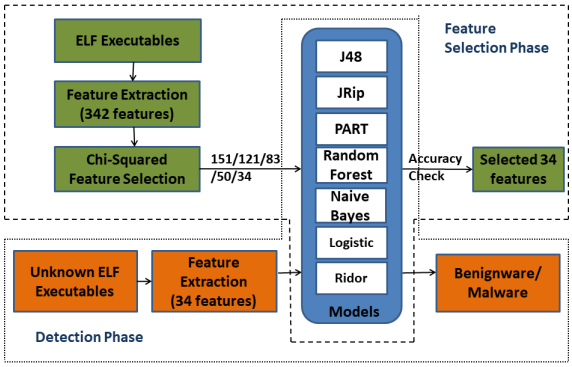


Fig. 1. Block Diagram of the proposed framework

Initially, 383 features which were used in the ELF-Miner are extracted from the ELF executables. Then we eliminated those features which have garbage, constant or null value throughout the dataset and reduced the number of features to 342. But, not all features are important in distinguishing between malware and benign. Some features will have less correlation with the output class of the executable. Including those features may have a negative impact on the prediction of the classifier. Also, since the proposed framework is aimed at resource-constrained devices, the time taken to detect the malware should be reduced. Hence we tried different feature selection techniques like information gain and chi-square on our dataset. Chi-square technique was able to reduce the number of features significantly, with less impact on the accuracy. Hence, we have deployed a feature selection phase which uses chi-square technique (see section III A). Here, when the whole dataset is input to the chi-square algorithm, it will give the 342 features ranked according to their importance in determining the class label. Then, we set different thresholds for the chi-square statistic empirically and kept only the features having value higher than threshold in the dataset. Machine learning models like J48, JRip, PART, Random Forest, Naive Bayes, RIDOR, and Logistic were trained on this dataset to analyse the impact on accuracy. Finally, selected 34 highest rank features with the highest chi-square value (here 3196) which are highly dependent on the class label and having near optimal accuracy. The 34 features are given in Table I and the description is available in [29].

TABLE I
34 FEATURES USED IN OUR APPROACH

| No | Feature | No | Feature |
|---|---|---|---|
| 1 | STT_OBJECT_STB_GLOBAL | 18 | DYNINIT |
| 2 | DYNRELA | 19 | DYNINIT_ARRAY |
| 3 | DYNPLTRELSZ | 20 | DYNINIT_ARRAYSZ |
| 4 | DYNRELASZ | 21 | DYNJMPREL |
| 5 | DYNRELAENT | 22 | DYNSTRTAB |
| 6 | STT_OBJECT | 23 | DYNSYMENT |
| 7 | STT_OBJECT_STB_WEAK | 24 | DYNSYMTAB |
| 8 | DYNSTRSZ | 25 | STB_LOCAL |
| 9 | STT_NOTYPE_STB_LOCAL | 26 | STB_WEAK |
| 10 | STT_NOTYPE_STB_WEAK | 27 | DYNVERNEED |
| 11 | DYNPLTREL | 28 | STT_FUNC_STB_GLOBAL |
| 12 | DYNPLTGOT | 29 | STB_GLOBAL |
| 13 | DYNNULL | 30 | STT_FUNC |
| 14 | DYNFINI_ARRAY | 31 | DYNDEBUG |
| 15 | DYNFINI | 32 | DYNVERNEEDNUM |
| 16 | DYNNEEDED | 33 | STT_FUNC_STB_WEAK |
| 17 | DYNFINI_ARRAYSZ | 34 | STT_NOTYPE |

In the training phase, we will use the final machine learning models trained with 34 features. In the detection phase, feature set extracted from the unknown executable is given to the 7 classifiers, which classify the executable as benign or malicious. The feature selection method and the classification models used are discussed in the following sections.

### A. Chi-square Feature Selection Method

Chi-square is used to determine the amount of dependence between a feature and the class label. It is based on comparing the obtained values of the frequency of a class because of the split to the expected frequency of the class [30]. Chi-square score with 'C' class and 'I' number of intervals is defined as:

$$X^2 = \sum_{i=1}^{C} \sum_{j=1}^{I} \frac{(N_{ij} - E_{ij})^2}{E_{ij}} \quad (1)$$

$N_{ij}$: number of samples of the $C_i$ class within the $j^{th}$ interval.

Expected frequency of $N_{ij}$ is given by:

$$E_{ij} = \frac{M_{Ij}.|C_i|}{N} \tag{2}$$

$M_{Ij}$: number of samples in the $j^{th}$ interval.

N: number of samples.

When two features are independent, the observed value will be close to the expected value and thus will have a smaller chi-square value. A high value of chi-square will indicate that the feature is more dependent on the response variable i.e., class. Hence, features with high chi-square value should be used for training the model since those features will be more significant in distinguishing between the output classes. We have chosen chi-square statistic due to the following reasons.

- Reduce the features significantly without heavily reducing the accuracy.
- Make no assumptions (like normality) about the data distribution
- Easy to compute
- Can be used for categorical features.

Table II shows the reduction in the number of features when the threshold of chi-square statistic is varied from 1000 to 3196 (highest).

TABLE II
REDUCING THE FEATURES BY VARYING CHI-SQUARE VALUE THRESHOLD

| Chi-square Statistic | Number of Features |
|---|---|
| 1000-3196 | 151 |
| 2000-3196 | 121 |
| 3000-3196 | 83 |
| 3190-3196 | 50 |
| 3196 | 34 |

### B. Classification Models

A brief overview of the classification models used for Linux malware detection is given below:

1) J48: It is an open-source Java implementation of the C4.5 algorithm in the WEKA [31]. It basically creates a decision tree where each internal node will check the value of a set of features, each branch will be the outcome of the check and leaves will denote the class.
2) JRip: It is an implementation of RIPPER which is a propositional rule-based learner. RIPPER extracts a set of rules to classify the training data. Basically, it start with an initial model and iteratively improve it using heuristic techniques [32].
3) PART: It build a partial decision tree, an ordinary decision tree that contains branches to undefined subtrees. It does not perform global optimization like C4.5 and RIPPER to produce the appropriate rules [33].
4) Random Forest: It is an ensemble learning method which uses many decision trees. Each tree will make an individual prediction of the output class and the final prediction of the ensemble classifier will be based on the majority of votes of individual trees. The generalization error for forests converges to a limit as the number of trees in the forest becomes large [34].
5) Naive Bayes: It applies to learning tasks where each instance 'x' is described by a conjunction of attribute values and where the target function f(x) can take on any value from some finite set V [35]. It will assume that the features are independent and predict the class label for the new instance, based on the training data.
6) Logistic: Basically, it uses a logistic function to model a binary class label. Since the malware detection is a binary classification technique, logistic model can be used to classify the executables.
7) RIDOR: It constructs the default rule. An incremental reduced error pruning is used to find exceptions with the smallest error rate, finding the best exceptions for each exception and iterating [36].

We have used a combination of different classification models to assess their performance in accurately classifying a given executable as benign or malware. We have used probabilistic models like Naive Bayes and Logistic, decision model like J48, ensemble method like Random Forest and rule-based models like JRip, RIDOR, and PART.

## IV. EXPERIMENTS AND RESULTS

The malware dataset used in this paper was collected from VirusShare malware repository [37] which consists of 1773 new malicious ELF binaries collected since 2019. 1426 benign executables were collected from the Linux directories */bin*, */sbin*, */usr/bin*, and */usr/local/bin*.

Training of machine learning models was conducted on an Ubuntu 20.10 LTS operating system with 4GB RAM and 200GB storage. 342 features were extracted from different sections of the Linux ELF executables using *pyelftools* python library. Then, we have used the feature selection algorithm *ChiSquaredAttributeEval* with *Ranker search* method in Wakaito Environment for Knowledge Acquisition (WEKA) 3.6 toolkit and varied the chi-square statistic value thresholds from 1000 to 3196. The number of features obtained were 151, 121, 83, 50, and 34. For each set of features, we have used the standard implementations of the 7 machine learning classifiers J48, JRip, PART, Random Forest, Naive Bayes, Logistic, and RIDOR in WEKA. Since the proposed malware detection approach focuses on IoT devices, prediction of the unknown executable i.e., detection phase needs to be implemented on a resource-constrained environment. Hence, the classification of unknown executable into malware/benignware was conducted on an Ubuntu 20.10 LTS operating system with 1GB RAM and 20GB storage. This configuration is used to mimic a relatively small gateway device such as Raspberry Pi.

In our experiments, we have considered the Linux malware detection as a binary classification model where malware denotes the positive class and benign denotes the negative class. For all the experiments, we have partitioned the dataset with 3196 instances into 70% training and 30% test. We have used the metrics like accuracy, false positive rate (FPR), and

false negative rate (FNR) to evaluate the efficiency of the malware detection model. Accuracy is the ratio of the correct predictions to the total number of instances in the dataset. Here malware denotes positive class. Since our approach is aimed at resource-constrained devices, reducing the false positive rate is of great importance. If a benign file is marked as malicious, alarm will be raised and a lot of resources will be wasted to check whether the file is actually malicious or it can prevent a benign file from performing its functionality. Also, a high false negative rate can cause very serious repercussions on the system, since a malware is allowed to run thinking it as a benign file.

### A. Results

Table III shows the performance metrics of the 7 classification models for each set of features. JRip corresponds to the Java implementation of RIPPER classifier in WEKA.

TABLE III
PERFORMANCE METRICS OF THE 7 CLASSIFIERS FOR EACH FEATURE SET

| Models used | | Number of features | | | | |
|---|---|---|---|---|---|---|
| | | 151 | 121 | 83 | 50 | 34 |
| J48 | Accuracy | 100 | 100 | 100 | 99.58 | 99.58 |
| | FPR | 0 | 0 | 0 | 0.005 | 0.005 |
| | FNR | 0 | 0 | 0 | 0.004 | 0.004 |
| JRip | Accuracy | 99.90 | 99.90 | 99.90 | 99.69 | **99.69** |
| | FPR | 0 | 0 | 0 | 0.005 | 0.005 |
| | FNR | 0.002 | 0.002 | 0.002 | 0.002 | **0.002** |
| PART | Accuracy | 100 | 100 | 100 | 99.58 | 99.06 |
| | FPR | 0 | 0 | 0 | 0.005 | 0.016 |
| | FNR | 0 | 0 | 0 | 0.004 | 0.004 |
| Random Forest | Accuracy | 100 | 100 | 100 | 99.69 | **99.69** |
| | FPR | 0 | 0 | 0 | 0.005 | 0.005 |
| | FNR | 0 | 0 | 0 | 0.002 | **0.002** |
| Naive Bayes | Accuracy | 99.79 | 99.90 | 100 | 97.81 | 97.81 |
| | FPR | 0.002 | 0.002 | 0 | 0.002 | **0.002** |
| | FNR | 0.002 | 0 | 0 | 0.038 | 0.038 |
| Logistic | Accuracy | 99.90 | 99.90 | 99.90 | 99.69 | **99.69** |
| | FPR | 0.002 | 0 | 0 | 0.005 | 0.005 |
| | FNR | 0 | 0.002 | 0.002 | 0.002 | **0.002** |
| RIDOR | Accuracy | 99.90 | 99.90 | 99.90 | 99.27 | 99.27 |
| | FPR | 0 | 0 | 0 | 0.005 | 0.005 |
| | FNR | 0.002 | 0.002 | 0.002 | 0.010 | 0.010 |

As can be seen from Table III, even after reducing the number of features to 34, the reduction in the accuracy is at most 2% for Naive Bayes and models like JRip, Logistic, and Random Forest achieve the highest accuracy of 99.69%. Also, there is no considerable increase in the false positive rate and false negative rate for all the models. As highlighted in Table III, JRip, Logistic, and Random Forest models perform slightly better than the other models. This proves the fact that the chi-square feature selection method managed to capture only those features that have significantly higher correlation with the classification label. Table IV shows the time taken to train the models for each set of features.

As shown in Table IV, we can reduce the time taken to train the JRip model by 0.25s, Random Forest model by 0.56s and Logistic model by 0.47s by reducing the number of features from 151 to 34. This time is very much significant when it comes to resource-constrained devices. On an average, feature extraction and preprocessing times per instance are 44.9 milliseconds and 0.16 milliseconds respectively. Extracting features from the unknown executable, preprocessing the features and predicting whether the file is malicious or not using the 7 trained machine learning models took ≈20 seconds and memory consumption was around 12 MB.

TABLE IV
TIME TAKEN (IN SECONDS) TO TRAIN THE MODEL FOR DIFFERENT SET OF FEATURES

| Models | Number of features | | | | |
|---|---|---|---|---|---|
| | 151 | 121 | 83 | 50 | 34 |
| J48 | 0.28 | 0.23 | 0.23 | 0.18 | 0.14 |
| JRip | 0.37 | 0.33 | 0.32 | 0.26 | **0.12** |
| PART | 0.1 | 0.08 | 0.08 | 0.007 | 0.05 |
| Random Forest | 1.24 | 1.07 | 0.78 | 0.7 | **0.68** |
| Naive Bayes | 0.21 | 0.1 | 0.09 | 0.07 | 0.04 |
| Logistic | 1.17 | 1.15 | 0.84 | 0.56 | **0.37** |
| RIDOR | 0.35 | 0.25 | 0.23 | 0.22 | 0.2 |

### B. Comparison with ELF-Miner

ELF-Miner dataset consists of 734 instances of benign executables and 709 instances of different categories of malware executables. They provide more than 99% detection accuracy with less than 0.1% false alarm rate [6]. The authors of ELF-Miner also have done a scalability analysis by extracting top 88 features which have an information gain of 0.1 or above. But in this paper, we have used a bigger dataset with 3196 instances. Even with such a bigger dataset, our proposed approach was able to attain an accuracy of around 99% for all the 7 classification models with only 34 features.

ELF-Miner was proposed for computer systems running Linux OS, whereas, our approach focuses on resource-constrained IoT devices. ELF-Miner have extracted 383 features and used information gain criteria to select the 187-202 features for preparing 8 different datasets corresponding to 8 malware categories. Information gain measures the reduction in the entropy that is achieved by learning a feature or variable [30]. On the other hand, our approach managed to select 34 features using chi-square statistic.

Table V compares the accuracy of the J48, PART, and RIPPER (JRip) classifiers for both the ELF-Miner and our proposed approach with 34 features, since those were the machine learning classifiers used in ELF-Miner for performance evaluation. As can be seen, there is only a slight decrease in the accuracy when we reduced the number of features to 34.

TABLE V
ACCURACY COMPARISON OF ELF-MINER AND OUR PROPOSED APPROACH

| Models | Accuracy | |
|---|---|---|
| | ELF-Miner | Proposed methodology |
| J48 | 99.98 | 99.58 |
| RIPPER | 100 | 99.69 |
| PART | 99.98 | 99.06 |

Table VI shows the variation in accuracy when the number of features are varied based on the feature selection criteria threshold, information gain for ELF-Miner and chi-square for our approach. In this paper, we proposed a framework which can predict whether a given Linux executable is malware or benignware without much loss in accuracy, within a considerably less time compared to ELF-Miner.

TABLE VI
EFFECT OF FEATURE SELECTION ON J48 CLASSIFIER FOR ELF-MINER AND OUR PROPOSED APPROACH

| ELF-Miner | | | Proposed Approach | | |
|---|---|---|---|---|---|
| Info-Gain | Features | Accuracy | Chi-square | Features | Accuracy |
| 0.10-0.99 | 88 | 100 | 2000-3196 | 121 | 100 |
| 0.10-0.79 | 77 | 99.58 | 3000-3196 | 83 | 100 |
| 0.10-0.69 | 66 | 99.58 | 3190-3196 | 50 | 99.58 |
| 0.10-0.49 | 42 | 99.02 | 3196 | 34 | 99.58 |

## V. CONCLUSION

In this paper, we have introduced a malware detection framework for resource-constrained systems such as IoT devices running Linux OS and apply 'chi-square' feature selection approach to determine the 34 features that are more significant in discriminating malware from benignware. Our approach achieved more than 99% accuracy with less than 0.1% false alarm rate for all the 7 classification models with approximately 3196 instances.

## REFERENCES


[1] McGraw, G. and Morrisett, G., 2000. 'Attacking malicious code: A report to the infosec research council.' IEEE software, 17(5), pp.33-41.

[2] Av-TEST Security Institute 2021, *'Malware'*, AVTEST, viewed 21 October 2021, ⟨https://www.av-test.org/en/statistics/malware/⟩

[3] Asmitha, K.A. and Vinod, P., 2014, February. 'A machine learning approach for linux malware detection.' In 2014 international conference on issues and challenges in intelligent computing techniques (ICICT) (pp. 825-830). IEEE.

[4] Damodaran, A., Di Troia, F., Visaggio, C.A., Austin, T.H. and Stamp, M., 2017. 'A comparison of static, dynamic, and hybrid analysis for malware detection.' Journal of Computer Virology and Hacking Techniques, 13(1), pp.1-12.

[5] Souri, A. and Hosseini, R., 2018. 'A state-of-the-art survey of malware detection approaches using data mining techniques.' Human-centric Computing and Information Sciences, 8(1), pp.1-22.

[6] Shahzad, F. and Farooq, M., 2012. 'Elf-miner: Using structural knowledge and data mining methods to detect new (linux) malicious executables.' Knowledge and information systems, 30(3), pp.589-612.

[7] Asmitha, K.A. and Vinod, P., 2014, September. 'Linux malware detection using non-parametric statistical methods.' In 2014 International Conference on Advances in Computing, Communications and Informatics (ICACCI) (pp. 356-361). IEEE.

[8] Krebs, B 2016, *'Hacked Cameras, DVRs Powered Today's Massive Internet Outage'*, KrebsonSecurity, viewed 12 August 2021, ⟨https://krebsonsecurity.com/2016/10/hacked-cameras-dvrs-powered-todays-massive-internet-outage/⟩

[9] Kumar, A. and Lim, T.J., 2019, April. 'EDIMA: early detection of IoT malware network activity using machine learning techniques.' In 2019 IEEE 5th World Forum on Internet of Things (WF-IoT) (pp. 289-294).

[10] Ngo, Q.D., Nguyen, H.T., Le, V.H. and Nguyen, D.H., 2020. 'A survey of IoT malware and detection methods based on static features.' ICT Express, 6(4), pp.280-286.

[11] KernelCare Team 2019, *'IoT Devices are in Desperate Need of Live Kernel Patching,'*, KernelCare, viewed 14 August 2021, ⟨https://blog.kernelcare.com/iot-devices-are-in-desperate-need-of-live-kernel-patching⟩

[12] Cozzi, E., Graziano, M., Fratantonio, Y. and Balzarotti, D., 2018, May. 'Understanding linux malware.' In 2018 IEEE symposium on security and privacy (SP) (pp. 161-175). IEEE.

[13] Raju, A.D., Abualhaol, I.Y., Giagone, R.S., Zhou, Y. and Huang, S., 2021. 'A Survey on Cross-Architectural IoT Malware Threat Hunting.' IEEE Access, 9, pp.91686-91709.

[14] Damri, G. and Vidyarthi, D., 2016, March. 'Automatic dynamic malware analysis techniques for Linux environment.' In 2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom) (pp. 825-830). IEEE.

[15] Shahzad, F., Shahzad, M. and Farooq, M., 2013. 'In-execution dynamic malware analysis and detection by mining information in process control blocks of Linux OS.' Information Sciences, 231, pp.45-63.

[16] Bai, J., Yang, Y., Mu, S. and Ma, Y., 2013. 'Malware detection through mining symbol table of Linux executables.' Information Technology Journal, 12(2), p.380.

[17] Kuzin, M., Shmelev, Y., Kuskov, V. 2018, *'New trends in the world of IoT threats.'*, SECURELIST by Kaspersky, viewed 1 September 2021, ⟨https://securelist.com/new-trends-in-the-world-of-iot-threats/87991/⟩

[18] Fouladi, M.M.A., 'Security in resource-constrained computer systems.'

[19] Podder, P., Mondal, M., Bharati, S. and Paul, P.K., 2021. 'Review on the security threats of internet of things.' arXiv preprint arXiv:2101.05614.

[20] Lee, Y.T., Ban, T., Wan, T.L., Cheng, S.M., Isawa, R., Takahashi, T. Inoue, D., 2020, December. 'Cross platform iot-malware family classification based on printable strings.' In 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) (pp. 775-784). IEEE.

[21] Wan, T.L., Ban, T., Cheng, S.M., Lee, Y.T., Sun, B., Isawa, R., Takahashi, T. and Inoue, D., 2020. 'Efficient detection and classification of internet-of-things malware based on byte sequences from executable files.' IEEE Open Journal of the Computer Society, 1, pp.262-275.

[22] Wan, T.L., Ban, T., Lee, Y.T., Cheng, S.M., Isawa, R., Takahashi, T. and Inoue, D., 2020, August. 'IoT-malware detection based on byte sequences of executable files.' In 2020 15th Asia Joint Conference on Information Security (AsiaJCIS) (pp. 143-150). IEEE.

[23] Tien, C.W., Chen, S.W., Ban, T. and Kuo, S.Y., 2020. 'Machine learning framework to analyze iot malware using elf and opcode features.' Digital Threats: Research and Practice, 1(1), pp.1-19.

[24] Sihwail, R., Omar, K. and Ariffin, K.A.Z., 2018. 'A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis.' International Journal on Advanced Science, Engineering and Information Technology, 8(4-2), p.1662.

[25] Kumar, R. and Geethakumari, G., 2019, October. 'Temporal Dynamics and Spatial Content in IoT Malware detection.' In TENCON 2019-2019 IEEE Region 10 Conference (TENCON) (pp. 1590-1595). IEEE.

[26] Jeon, J., Park, J.H. and Jeong, Y.S., 2020. 'Dynamic analysis for IoT malware detection with convolution neural network model.' IEEE Access, 8, pp.96899-96911.

[27] Dang, F., Li, Z., Liu, Y., Zhai, E., Chen, Q.A., Xu, T., Chen, Y. and Yang, J., 2019, June. 'Understanding fileless attacks on linux-based iot devices with honeycloud.' In Proceedings of the 17th Annual International Conference on Mobile Systems, Applications, and Services (pp.482-493)

[28] El Merabet, H. and Hajraoui, A., 2019. 'A survey of malware detection techniques based on machine learning.' Int. J. Adv. Comput. Sci. Appl, 10(1), pp.366-373.

[29] TIS Committee 1995, 'Tool interface standard (TIS) executable and linking format (ELF) specification version 1.2.'

[30] Sulistiani, H. and Tjahyanto, A., 2017. 'Comparative Analysis of Feature Selection Method to Predict Customer Loyalty.' IPTEK The Journal of Engineering, 3(1), pp.1-5.

[31] Gholap, J., 2012. 'Performance tuning of J48 Algorithm for prediction of soil fertility.' arXiv preprint arXiv:1208.3943.

[32] Cohen, W.W., 1995. 'Fast effective rule induction.' In Machine learning proceedings 1995 (pp. 115-123). Morgan Kaufmann.

[33] Frank, E. and Witten, I.H., 1998. 'Generating accurate rule sets without global optimization.'

[34] Breiman, L., 2001. 'Random forests.' Machine learning, 45(1), pp.5-32.

[35] Mitchell, T., 1997. 'Machine learning.'

[36] Veeralakshmi, V. and Ramyachitra, D., 2015. 'Ripple Down Rule learner (RIDOR) classifier for IRIS dataset.' Issues, 1(1), pp.79-85.

[37] VirusShare Malware Repository, viewed 18 June 2021, ⟨https://virusshare.com/⟩